

```

"""
AK SIGNAL BOT - premium dashboard
Run: streamlit run ak_signal_bot.py
"""

import requests
import pandas as pd
import streamlit as st
import plotly.graph_objects as go
import yfinance as yf

import ak_engine as eng

st.set_page_config(page_title="AK SIGNAL BOT",
page_icon="📈", layout="wide")

PAIRS = {
    "EUR/USD": "EURUSD=X", "GBP/USD":
"GBPUSD=X", "USD/JPY": "JPY=X",
    "AUD/USD": "AUDUSD=X", "USD/CAD": "CAD=X",
    "NZD/USD": "NZDUSD=X",
    "USD/CHF": "CHF=X", "EUR/GBP": "EURGBP=X",
    "EUR/JPY": "EURJPY=X",
    "GBP/JPY": "GBPJPY=X", "NZD/CAD":
"NZDCAD=X",
    "BTC/USD": "BTC-USD", "ETH/USD": "ETH-USD",
    "Gold": "GC=F",
}
TF = {"1 min": ("1m", "5d"), "5 min": ("5m",
"30d"),
      "15 min": ("15m", "30d"), "1 hour": ("1h",
"90d")}
TZ = "Asia/Karachi"

# -----

```

```

----- styling
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?
family=Sora:wght@500;700;800&family=DM+Sans:wght
@400;500;700&display=swap');
:root{--bg:#0a0e1a;--panel:#111729;--
line:#232c47;--gold:#d8b45a;--gold2:#f3dd9c;
--up:#27d3a2;--down:#ff5c72;--mute:#8d97b5;--
text:#eef1fb}
.stApp{background:radial-gradient(1200px 500px
at 15% -10%,#1a2347 0%,var(--bg) 60%);
color:var(--text);font-family:'DM Sans',sans-
serif}
header[data-testid="stHeader"]
{background:transparent}
section[data-testid="stSidebar"]
{background:#0c1122;border-right:1px solid var(--
line)}
h1,h2,h3{font-family:'Sora',sans-
serif!important;letter-spacing:-.01em}
.brand{display:flex;align-
items:center;gap:14px;margin:4px 0 18px}
.brand .mark{width:46px;height:46px;border-
radius:12px;display:grid;place-items:center;
background:linear-gradient(135deg,var(--
gold2),var(--gold) 55%,#9b7a2c);color:#1a1405;
font:800 18px 'Sora';box-shadow:0 8px 28px
rgba(216,180,90,.28)}
.brand .name{font:800 28px
'Sora';background:linear-gradient(90deg,var(--
gold2),var(--gold));
-webkit-background-clip:text;-webkit-text-fill-
color:transparent}
.brand .tag{color:var(--mute);font-
size:13px;margin-top:-2px}
.card{background:linear-

```

```

gradient(180deg,#131a30,#0f1527);border:1px
solid var(--line);
border-radius:18px;padding:22px 24px;margin-
bottom:16px}
.sig{display:flex;justify-content:space-
between;align-items:center;flex-
wrap:wrap;gap:18px}
.sig .word{font:800 64px/1 'Sora';letter-
spacing:-.02em}
.sig.CALL{border-color:rgba(39,211,162,.55);box-
shadow:0 0 0 1px rgba(39,211,162,.15),0 18px
60px rgba(39,211,162,.12)}
.sig.PUT{border-color:rgba(255,92,114,.55);box-
shadow:0 0 0 1px rgba(255,92,114,.15),0 18px
60px rgba(255,92,114,.12)}
.CALL .word{color:var(--up)}.PUT
.word{color:var(--down)}.WAIT .word{color:var(--
mute)}
.meta{color:var(--mute);font-size:14px;line-
height:1.7}
.meta b{color:var(--text);font-weight:700}
.bar{height:8px;border-
radius:99px;background:#1b2440;overflow:hidden;m
argin-top:8px;width:220px}
.bar i{display:block;height:100%;border-
radius:99px;background:linear-
gradient(90deg,var(--gold),var(--gold2))}
table.votes{width:100%;border-
collapse:collapse;font-size:14px}
table.votes td{padding:10px 8px;border-
bottom:1px solid var(--line)}
table.votes tr:last-child td{border-bottom:none}
.pill{display:inline-block;min-width:44px;text-
align:center;padding:3px 10px;border-
radius:99px;
font-weight:700;font-size:13px}
.p-

```

```

up{background:rgba(39,211,162,.14);color:var(--up)}
.p-
dn{background:rgba(255,92,114,.14);color:var(--down)}
.p-no{background:#1b2440;color:var(--mute)}
div.stButton>button{background:linear-gradient(135deg,var(--gold2),var(--gold)60%,#a98430);color:#1a1405;font:700 16px'Sora';border:0;border-radius:14px;padding:14px22px;width:100%;box-shadow:0 10px 30px rgba(216,180,90,.25)}
div.stButton>button:hover{filter:brightness(1.07);color:#1a1405}
.note{color:var(--mute);font-size:12.5px;line-height:1.6}
@media (max-width:640px){.sig .word{font-size:48px}}
</style>
""", unsafe_allow_html=True)

st.markdown("""
<div class="brand"><div class="mark">AK</div>
<div><div class="name">AK SIGNAL BOT</div>
<div class="tag">RSI, EMA, MACD, Stochastic,
Bollinger, Support and Resistance, candle
patterns</div></div></div>
""", unsafe_allow_html=True)

# -----
----- data
@st.cache_data(ttl=45, show_spinner=False)
def load(symbol, interval, period):
    df = yf.download(symbol, interval=interval,
period=period,

```

```

        progress=False,
auto_adjust=True)
    if df is None or df.empty:
        return pd.DataFrame()
    if isinstance(df.columns, pd.MultiIndex):
        df.columns =
df.columns.get_level_values(0)
    df = df[["Open", "High", "Low",
"Close"]].dropna()
    if df.index.tz is not None:
        df.index = df.index.tz_convert(TZ)
    return df.iloc[:-1] # drop the candle that
is still forming

def send_telegram(token, chat_id, text):
    try:
        r =
requests.post(f"https://api.telegram.org/bot{tok
en}/sendMessage",
                json={"chat_id":
chat_id, "text": text, "parse_mode": "HTML"},
                timeout=10)
        return r.ok, r.text
    except Exception as ex:
        return False, str(ex)

def tg_text(pair, tf, expiry, res):
    icon = {"CALL": "🟢 CALL (UP)", "PUT": "🔴
PUT (DOWN)", "WAIT": "⚪ WAIT"}[res["signal"]]
    return (f"🔔 <b>AK SIGNAL BOT</b>\n\n<b>
{pair}</b> | {tf}\n"
            f"Signal: <b>{icon}</b>\nStrength:
{res['strength']}%\n"
            f"Entry price:
{res['price']:.5g}\nExpiry: {expiry}

```

```

candle(s)\n"
        f"Candle time: {res['time']:%d %b
%H:%M}\n\n"
        f"<i>Not financial advice. Test on
demo first.</i>")

# -----
----- sidebar
with st.sidebar:
    st.markdown("### Settings")
    pair = st.selectbox("Trading asset",
list(PAIRS))
    tf = st.selectbox("Time frame", list(TF),
index=1)
    expiry = st.select_slider("Expiry
(candles)", [1, 2, 3, 5], value=1)
    thr = st.slider("Signal strictness", 3, 9,
5,
                    help="Higher means fewer but
stronger signals")
    payout = st.slider("Broker payout %", 70,
95, 85) / 100
    st.markdown("### Telegram alerts")
    tg_on = st.toggle("Send signal to Telegram")
    tg_token = st.text_input("Bot token",
type="password") if tg_on else ""
    tg_chat = st.text_input("Chat ID") if tg_on
else ""
    st.markdown('<p class="note">Data comes from
real market feeds. '
                'Quotex OTC prices are not
available outside Quotex.</p>',
                unsafe_allow_html=True)

interval, period = TF[tf]

```

```

def votes_html(res):
    rows = ""
    for v in res["votes"]:
        cls = "p-up" if v["vote"] > 0 else "p-
dn" if v["vote"] < 0 else "p-no"
        rows += (f"<tr><td><b>{v['name']}</b>
</td><td class='meta'>{v['note']}</td>"
                f"<td style='text-align:right'>
<span class='pill {cls}'>{v['vote']:+d}</span>
</td></tr>")
    return f"<table class='votes'>{rows}
</table>"

def chart(res, pair):
    d = res["df"].tail(90)
    f = go.Figure()
    f.add_trace(go.Candlestick(x=d.index,
open=d.Open, high=d.High, low=d.Low,
close=d.Close,

increasing_line_color="#27d3a2",
decreasing_line_color="#ff5c72",
                        name="Price"))
    for col, color in (("ema9", "#f3dd9c"),
("ema21", "#6aa9ff"), ("ema50", "#b78cff")):
        f.add_trace(go.Scatter(x=d.index,
y=d[col], line=dict(width=1.3, color=color),
name=col.upper()))
    f.add_trace(go.Scatter(x=d.index, y=d.bb_up,
line=dict(width=.8, color="#4b5679",
dash="dot"), name="BB"))
    f.add_trace(go.Scatter(x=d.index,
y=d.bb_low, line=dict(width=.8, color="#4b5679",
dash="dot"), showlegend=False))
    price = res["price"]

```

```

        for lv, touches in res["levels"]:
            col = "#27d3a2" if lv <= price else
"#ff5c72"
            f.add_hline(y=lv, line=dict(color=col,
width=1, dash="dash"), opacity=.55)
            f.update_layout(height=480, margin=dict(l=8,
r=8, t=30, b=8),
xaxis_rangeslider_visible=False,

paper_bgcolor="rgba(0,0,0,0)",
plot_bgcolor="#0f1527",
                        font=dict(color="#c9d1ea"),
title=f"{pair} {tf}",
                        legend=dict(orientation="h",
y=1.08),

xaxis=dict(gridcolor="#1b2440"),
yaxis=dict(gridcolor="#1b2440"))
        return f

# -----
----- main
tab1, tab2, tab3 = st.tabs(["Signal", "Market
scanner", "Backtest"])

with tab1:
    go_btn = st.button("Generate signal")
    if go_btn:
        with st.spinner("Reading the
market..."):
            df = load(PAIRS[pair], interval,
period)
            if df.empty or len(df) < 80:
                st.error("No data received. Markets
may be closed (weekend for forex), "
                        "or the internet is down.

```

```

Try a crypto pair like BTC/USD.")
    else:
        res = eng.analyze(df, threshold=thr)
        st.session_state["res"] = res
        if tg_on and tg_token and tg_chat:
            ok, msg =
send_telegram(tg_token, tg_chat, tg_text(pair,
tf, expiry, res))
            st.success("Sent to Telegram")
if ok else st.error(f"Telegram error: {msg}")
        res = st.session_state.get("res")
        if res:
            s, r_ = res["support"],
res["resistance"]
            sup = f"{s[0]:.5g}" if s else "none
found"
            rsn = f"{r_[0]:.5g}" if r_ else "none
found"
            label = {"CALL": "CALL, price likely
up", "PUT": "PUT, price likely down",
                    "WAIT": "WAIT, no clear setup"}
[res["signal"]]
            st.markdown(f"""
<div class="card sig {res['signal']}">
  <div><div class="word">{res['signal']}</div>
  <div class="meta">{label}</div>
  <div class="bar"><i style="width:
{max(res['strength'],4)}%"></i></div>
  <div class="meta">Signal strength <b>
{res['strength']}%</b></div></div>
  <div class="meta">Asset <b>{pair}</b> on <b>
{tf}</b><br>
  Entry price <b>{res['price']:.5g}</b><br>
  Expiry <b>{expiry}</b> candle(s)<br>
  Candle time <b>{res['time']:%d %b, %H:%M}</b>
(Pakistan)<br>
  Nearest support <b>{sup}</b>, resistance <b>

```

```

{rsn}</b></div>
</div>""", unsafe_allow_html=True)
    c1, c2 = st.columns([1, 1.6])
    with c1:
        st.markdown("#### Indicator votes")
        st.markdown(f"<div class='card'>
{votes_html(res)}</div>",
unsafe_allow_html=True)
    with c2:
        st.plotly_chart(chart(res, pair),
use_container_width=True)
    else:
        st.markdown("<div class='card meta'>Pick
an asset and time frame, then press "
                    "<b>Generate signal</b>."
                    "</div>", unsafe_allow_html=True)

with tab2:
    st.markdown("Scan every asset on your chosen
time frame and list the strongest setups.")
    if st.button("Scan all assets"):
        rows = []
        bar = st.progress(0.0)
        for i, (name, sym) in
enumerate(PAIRS.items()):
            df = load(sym, interval, period)
            if not df.empty and len(df) >= 80:
                r = eng.analyze(df,
threshold=thr)
                rows.append({"Asset": name,
"Signal": r["signal"], "Strength %":
r["strength"],
                        "Score":
r["score"], "Price": round(r["price"], 5)})
                bar.progress((i + 1) / len(PAIRS))
        bar.empty()
        if rows:

```

```

        out =
pd.DataFrame(rows).sort_values("Strength %",
ascending=False)
        st.dataframe(out, hide_index=True,
use_container_width=True)
    else:
        st.warning("No data. Forex markets
are closed on weekends; try again on a
weekday.")

with tab3:
    st.markdown("Check how this strategy would
have performed on recent history before using
it.")
    if st.button("Run backtest"):
        with st.spinner("Testing on past
candles..."):
            df = load(PAIRS[pair], interval,
period)
            if df.empty or len(df) < 120:
                st.error("Not enough data for a
backtest.")
            else:
                bt = eng.backtest(df,
expiry=expiry, threshold=thr, payout=payout)
                a, b, c = st.columns(3)
                a.metric("Trades", bt["trades"])
                b.metric("Win rate", f"
{bt['win_rate']:.1f}%")
                c.metric("Break-even needed", f"
{bt['break_even']:.1f}%")
                good = bt["trades"] >= 30 and
bt["win_rate"] > bt["break_even"] + 3
                if bt["trades"] < 30:
                    st.warning("Too few trades
to trust this result. Try a lower strictness or
another asset.")

```

```
        elif good:
            st.success(f"Above break-
even on this sample (net {bt['net_units']:+.1f}
stakes). "
                    "Past results do
not guarantee the future. Test on demo.")
        else:
            st.error(f"Not profitable on
this sample (net {bt['net_units']:+.1f} stakes).
"
                    "Do not trade this
setting with real money.")

st.markdown("<p class='note'>Signals are
indicator calculations, not predictions or
financial advice. "
            "Binary options carry a high risk of
losing your stake. Use a demo account first.
</p>",
            unsafe_allow_html=True)
```